

Simple Command Sequencer Generator

Autocoder User Guide

1 Description

The purpose of this generator is to provide a user-friendly way of defining named command sequences for the Simple Command Sequencer component. The generator takes a YAML model file as input which declares a suite of sequences – each a named, ordered list of sub-commands (and optional sleeps) – and autocodes four things from it: the Ada sequence step tables, bundled with the suite’s frame-pool size into a `Config` constant consumed by the sequencer’s `Init` routine; a set of first-class per-sequence wrapper commands so that each sequence appears on the sequencer instance as its own typed, operator-callable command; an operator-side command builder package for those wrapper commands; and the record type of the instance’s periodic summary packet.

Note the example shown in this documentation is used in the unit test of this component so that the reader of this document can see it being used in context. Please refer to the unit test code for more details on how this generator can be used.

2 Schema

The following pykwalify schema is used to validate the input YAML model. Model files must be named in the form *optional_name.assembly_name.command_sequences.yaml*, where *optional_name* is the specific name of this set of sequences (only needed if more than one suite is defined in the assembly) and *assembly_name* is the assembly the sequences will be used in. Generally this file is created in the same directory or near to the assembly model file. The schema is commented to show each available YAML key and what it accomplishes; even without knowing the specifics of pykwalify schemas, you should be able to glean the available structure from the file below.

```
1  # Schema for command_sequences YAML files
2  # This defines the structure and validation rules
3
4  type: map
5  mapping:
6    name:
7      required: true
8      type: str
9      desc: |
10         The name of the command sequences package. This will be used as the
11         Ada package name, so it must follow Ada naming conventions.
12
13    description:
14      required: false
15      type: str
16      desc: |
17         Description of this command sequences package. This will appear in
18         the generated Ada code comments.
19
20    preamble:
21      required: false
22      type: str
```

```

23     desc: |
24         Custom preamble text to include in the generated Ada package.
25
26 num_concurrent_sequences:
27     required: true
28     type: int
29     range: { min: 1 }
30     desc: |
31         The number of sequence frames (concurrently running sequences) a
32         sequencer instance initialized with this suite allocates. This also
33         sizes the suite's generated summary packet ground type (one
34         Sequence_Frame_Summary per frame), so the frame pool and the packet
35         layout can never disagree. The upper bound -- how many frame
36         summaries fit in one packet buffer -- is enforced by the model
37         against the project configuration.
38
39 command_timeout_seconds:
40     required: false
41     type: int
42     range: { min: 0, max: 2147483 }
43     desc: |
44         Default timeout, in whole seconds, that the sequencer will wait for
45         a sub-command's response before declaring the sequence failed. Used
46         for any sequence in this suite that does not specify its own
47         command_timeout_seconds. Bounded so the millisecond count always
48         fits an Ada.Real_Time.Time_Span.
49
50 with:
51     required: false
52     type: seq
53     desc: |
54         List of additional Ada packages to "with" in the generated sequences
55         package. Use this when a step's arg expression references a package
56         that is not pulled in automatically, e.g. an enum package for a
57         qualified enumeration literal.
58     sequence:
59         - type: str
60
61 sequences:
62     required: true
63     type: seq
64     desc: |
65         List of command sequence definitions. Each sequence defines a named
66         command that executes a series of sub-commands.
67     sequence:
68         - type: map
69           mapping:
70             name:
71                 required: true
72                 type: str
73                 desc: |
74                     Name of this sequence. This will become a command that can be
75                     executed. Must follow Ada naming conventions.
76
77             description:
78                 required: false
79                 type: str
80                 desc: |
81                     Description of what this sequence does. Will appear in generated
82                     code and documentation.
83

```

```

84     arg_type:
85         required: false
86         type: str
87         desc: |
88             The Ada type for this sequence's argument. If not specified,
89             the sequence takes no arguments. Must be a valid Ada type name,
90             possibly qualified (e.g., "Power_Mode.T" or "Packed_U16.T").
91
92     wait_for_command_completion:
93         required: false
94         type: bool
95         default: true
96         desc: |
97             Default setting for whether to wait for each sub-command to
98             complete before executing the next one. Individual steps can
99             override this.
100
101     continue_on_failure:
102         required: false
103         type: bool
104         default: false
105         desc: |
106             If true, the sequence will continue executing even if a
107             ↪ sub-command
108             fails. If false, the sequence will abort on the first failure.
109
110     response_behavior:
111         required: false
112         type: str
113         enum: ['send_after_sequence_start',
114             ↪ 'send_after_sequence_completion']
115         desc: |
116             When the sequencer replies to this sequence's own command:
117             send_after_sequence_start (the default) acknowledges as soon as
118             the sequence is started; send_after_sequence_completion defers
119             the reply until the sequence finishes, carrying its final
120             success or failure. This is static per-sequence configuration.
121             Invocations through the generic Run_Sequence command choose
122             their behavior per-call in the command argument instead.
123
124     command_timeout_seconds:
125         required: false
126         type: int
127         range: { min: 0, max: 2147483 }
128         desc: |
129             Override the suite-level default timeout for sub-command
130             responses, in whole seconds, for this sequence only. Bounded
131             so the millisecond count always fits an
132             Ada.Real_Time.Time_Span.
133
134     sequence:
135         required: true
136         type: seq
137         desc: |
138             The ordered list of steps to execute. Each step is either a
139             command dispatch (`command:` + optional `arg:`) or an in-sequence
140             sleep (`sleep_ms:` + integer milliseconds). The two forms are
141             mutually exclusive; exactly one must be present per step.
142             Validation is enforced in the model loader.
143         sequence:
144             - type: map

```

```

mapping:
  command:
    required: false
    type: str
    desc: |
      The command to execute in the format
      ↪ "Component.Command_Name".
      The component must be a valid component instance name in
      ↪ your
      assembly, and the command must exist on that component.
      Mutually exclusive with `sleep:`.

  arg:
    required: false
    type: str
    desc: |
      An Ada expression that evaluates to the argument for this
      command. You can reference the parent sequence's argument
      using "Arg.field_name" syntax. If the command takes no
      argument, omit this field. Only valid alongside
      ↪ `command:`.

      Examples:
      - "Arg" - pass through the entire sequence argument
      - "Arg.Power_Mode" - pass a field from the sequence
      ↪ argument
      - "(Mode => Arg.Mode, Enable => True)" - construct a
      ↪ record
      - "42" - a literal value

  wait_for_completion:
    required: false
    type: bool
    desc: |
      Whether to wait for this specific command to complete
      ↪ before
      proceeding. If not specified, uses the parent sequence's
      wait_for_command_completion setting. Only valid alongside
      `command:`.

  sleep_ms:
    required: false
    type: text
    desc: |
      Milliseconds to pause the sequence. Not bus-routed; the
      sequencer frame parks itself for the duration without
      issuing any command. Mutually exclusive with
      `command:`/`arg:`/`wait_for_completion:`.

      Either a static integer (bounded to 0 .. 2**31 - 1 so it
      always fits an Ada.Real_Time.Time_Span; enforced by the
      model), or a dynamic reference into the sequence's
      argument using the same "Arg[field]..." syntax as
      dynamic command args. A dynamic sleep's referenced field
      must be a Packed_U32.T; its value is range-checked at
      execution time.

      Examples:
      - sleep_ms: 3000 # sleep 3 seconds
      - sleep_ms: Arg.Settle_Time # duration from the sequence
      ↪ argument

```

```

197
198 # Additional validation rules (enforced in the model Python code):
199 # - Command format must be "Component.Command" with valid identifiers, or
200 #   the reserved name "Sleep"
201 # - Referenced types in arg_type must exist in the system
202 # - Argument expressions must be valid Ada syntax
203 # - Component and command names must match instances in the assembly

```

3 Example Input

The following is an example command sequences input YAML file. It defines eleven sequences exercising the main features: statically-armed commands and static sleeps, typed argument records whose fields are dispatched as dynamic arguments to sub-commands (including through non-byte-aligned packed fields and validatable records), a dynamic sleep whose duration comes from the sequence argument, no-wait direct dispatch, a sequence that calls another sequence as a sub-sequence step, and sequences whose command response is statically deferred until completion. This example adheres to the schema shown in the previous section, and is commented to give clarification.

Note the optional top-level `with:` key, which lists additional Ada packages to “with” in the generated package. A step’s `arg:` expression is emitted verbatim into the generated Ada, so any package it references that is not pulled in automatically – such as an enum package needed to qualify an enumeration literal – must be listed here.

```

1  # Command Sequences Definition
2  # This file defines composite commands that execute sequences of sub-commands
3
4  name: Flight_Sequences
5  description: Common flight software command sequences
6
7  # Two sequence frames: sequences can run on both concurrently, and the
8  # generated summary packet type carries two frame summaries.
9  num_concurrent_sequences: 2
10
11 command_timeout_seconds: 10 # default wait for sub-command responses
12
13 sequences:
14   # Simple sequence with no arguments
15   - name: Sequence_A
16     description: Execute commands on component A
17     wait_for_command_completion: true
18     continue_on_failure: true
19
20     sequence:
21       - command: Component_A.Command_1
22       - command: Component_A.Command_2
23         arg: (Seconds => 3, Subseconds => 14)
24       - sleep_ms: 3000
25       - command: Component_A.Command_3
26         arg: (Value => 3)
27
28   - name: Sequence_B
29     description: Perform generic arguments with component A and B
30     wait_for_command_completion: true
31     continue_on_failure: false
32     arg_type: Sequence_B_Arg.T
33
34     sequence:
35       - command: Component_A.Command_3

```

```

36     arg: Arg.Component_A_Arg
37     - command: Component_B.Command_3
38     arg: Arg.Component_B_Arg
39
40 - name: Sequence_C
41   description: No-wait sequence for direct-dispatch testing
42   wait_for_command_completion: false
43   continue_on_failure: false
44
45   sequence:
46     - command: Component_A.Command_1
47     - command: Component_A.Command_1
48
49 - name: Sequence_D
50   description: Sequence with a dynamic sleep duration taken from the sequence
51   ↪ argument
52   wait_for_command_completion: true
53   continue_on_failure: false
54   arg_type: Packed_U32.T
55
56   sequence:
57     - sleep_ms: Arg
58
59   # A sequence that calls another sequence. Each sequence is a first-class
60   # command on the sequencer, so a sub-sequence step is just a command step
61   # targeting the sequencer's own synthesized command. The sub-sequence's
62   # response behavior is its own static configuration (Sequence_C defaults to
63   # send_after_sequence_start, so its reply arrives as soon as it starts).
64
65 - name: Sequence_E
66   description: Call Sequence_C as a sub-sequence, then run a regular command
67   wait_for_command_completion: true
68   continue_on_failure: false
69
70   sequence:
71     - command: Sequencer.Sequence_C
72     - command: Component_A.Command_1
73
74   # A sequence whose command response is statically deferred until the
75   # sequence completes -- the operator's reply carries the final
76   # success/failure instead of a start acknowledgement.
77
78 - name: Sequence_F
79   description: No-wait sequence with a statically deferred command response
80   wait_for_command_completion: false
81   continue_on_failure: false
82   response_behavior: send_after_sequence_completion
83
84   sequence:
85     - command: Component_A.Command_1
86
87   # A deferred sequence that waits on its sub-command, so the operator's
88   # deferred reply arrives only when the sub-command completes (or the
89   # sequence aborts, times out, or is killed).
90
91 - name: Sequence_G
92   description: Waiting sequence with a statically deferred command response
93   wait_for_command_completion: true
94   continue_on_failure: false
95   response_behavior: send_after_sequence_completion
96
97   sequence:
98     - command: Component_A.Command_1

```

```

96  # Regression coverage for dynamic-arg repacking: the sequence argument's
97  # leading 4-bit pad puts both leaf fields at non-byte-aligned offsets. Each
98  # step's resolver must deserialize the packed argument and re-serialize the
99  # leaf field standalone -- byte-offset slicing would corrupt both.
100 - name: Sequence_H
101   description: Dynamic steps traversing misaligned 8-bit and 16-bit packed
102   ↪ fields
103   wait_for_command_completion: true
104   continue_on_failure: false
105   arg_type: Nibble_Test_Args.T
106
107   sequence:
108     - command: Component_A.Command_8
109       arg: Arg.Wide
110     - command: Component_A.Command_7
111       arg: Arg.Nibble
112
113   # A sequence whose FIRST step is a static sleep, so wake-time overflow (the
114   # only failure a model-bounded static sleep has left) can be exercised
115   # without a command step's response deadline overflowing first.
116 - name: Sequence_I
117   description: A lone static sleep, for wake-time overflow coverage
118   wait_for_command_completion: true
119   continue_on_failure: false
120
121   sequence:
122     - sleep_ms: 3000
123
124   # The two sequences below take Validated_Args, whose Mode enumeration can
125   # fail validation -- exercising the invalid-dynamic-argument failure paths
126   # (all other test argument types accept every bit pattern).
127 - name: Sequence_J
128   description: Dynamic command argument resolved from a validatable record
129   wait_for_command_completion: true
130   continue_on_failure: false
131   arg_type: Validated_Args.T
132
133   sequence:
134     - command: Component_A.Command_3
135       arg: Arg.Duration
136
137 - name: Sequence_K
138   description: Dynamic sleep resolved from a validatable record
139   wait_for_command_completion: true
140   continue_on_failure: false
141   arg_type: Validated_Args.T
142
143   sequence:
144     - sleep_ms: Arg.Duration

```

4 Example Output

The example input above produces the following Ada outputs. The package spec declares the `To_Arg` buffer-padding helper, the per-dynamic-step Resolver functions, a step array constant per sequence, the `Sequences_Table` constant (plus a stable `Sequences` access pointer), and the suite's `Config` constant, which bundles the table access with the suite's `num_concurrent_sequences`. The package body implements the Resolver functions: each validates the sequence's per-call argument buffer through the input type's generated `Validation.Valid`, deserializes it through `Serialization.From_Byte_Array`, and re-serializes the addressed field as the step's argument

buffer (a sub-command's argument, or a Packed_U32 millisecond count for a dynamic sleep). Resolvers exist only for dynamic steps – a suite of purely static sequences declares none, and its body degenerates to `pragma No_Body;`.

The `Config` constant should be passed into the Simple Command Sequencer's `Init` procedure during assembly initialization, e.g. in the assembly YAML:

```
- type: Simple_Command_Sequencer
  name: My_Sequencer
  init:
    - "Config => Test_Assembly_Command_Sequences_Example_Sequences.Config"
```

```
1  -- Auto-generated by Adamant command sequences generator.
2  -- Do not edit by hand.
3  -- Common flight software command sequences
4  with Simple_Sequencer_Types; use Simple_Sequencer_Types;
5  with Test_Assembly_Commands; use Test_Assembly_Commands;
6  with Command_Types;
7  with Basic_Types;
8  with Sequence_Enums;
9  with Ada.Real_Time;
10 with Sys_Time_32;
11 with Packed_U32;
12 with Sequence_B_Arg;
13 with Nibble_Test_Args;
14 with Packed_U16;
15 with Packed_Nibble;
16 with Validated_Args;
17 package Test_Assembly_Command_Sequences_Example_Sequences is
18   -- Make the "&" operator on Byte_Array visible for To_Arg below.
19   use type Basic_Types.Byte_Array;
20
21   -----
22   -- Pad a serialized argument out to a full command argument buffer. An
23   -- expression function so the step-array constants below (elaborated with
24   -- this spec) can call it without an access-before-elaboration failure.
25   -----
26   function To_Arg (Bytes : Basic_Types.Byte_Array) return
27     ⇨ Command_Types.Command_Arg_Buffer_Type is
28     (Command_Types.Command_Arg_Buffer_Type (Bytes & Basic_Types.Byte_Array'(1
29       ⇨ .. Command_Types.Command_Arg_Buffer_Type'Length - Bytes'Length =>
30       ⇨ 0)));
31   -----
32   -- Resolver types - one per dynamic step, each encodes a traversal path.
33   -----
34   function Sequence_B_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
35     ⇨ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
36   function Sequence_B_Step_1_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
37     ⇨ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
38   function Sequence_D_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
39     ⇨ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
40   function Sequence_H_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
41     ⇨ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
42   function Sequence_H_Step_1_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
43     ⇨ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
44   function Sequence_J_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
45     ⇨ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
46   function Sequence_K_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
47     ⇨ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
48   -----
49   -- Step arrays - one constant per sequence.
50   -----
```



```

41 Sequence_A_Steps : aliased constant Step_Array :=
42 [
43     0 =>
44         (Kind      => Command_Step,
45          Id        => Component_A_Command_1,
46          Arg_Length => 0,
47          Arg        => [others => 0]),
48     1 =>
49         (Kind      => Command_Step,
50          Id        => Component_A_Command_2,
51          Arg_Length => Sys_Time_32.Serialization.Serialized_Length,
52          Arg        => To_Arg (Sys_Time_32.Serialization.To_Byte_Array
53                               ↪ ((Seconds => 3, Subseconds => 14))))),
53     2 =>
54         (Kind      => Sleep,
55          Id        => 0,
56          Arg_Length => 0,
57          Sleep_Arg  => 3000),
58     3 =>
59         (Kind      => Command_Step,
60          Id        => Component_A_Command_3,
61          Arg_Length => Packed_U32.Serialization.Serialized_Length,
62          Arg        => To_Arg (Packed_U32.Serialization.To_Byte_Array ((Value
63                               ↪ => 3))))),
63 ];
64 Sequence_B_Steps : aliased constant Step_Array :=
65 [
66     0 =>
67         (Kind      => Runtime_Argument_Command_Step,
68          Id        => Component_A_Command_3,
69          Arg_Length => Packed_U32.Serialization.Serialized_Length,
70          Resolver   => Sequence_B_Step_0_Resolver_T'Access),
71     1 =>
72         (Kind      => Runtime_Argument_Command_Step,
73          Id        => Component_B_Command_3,
74          Arg_Length => Packed_U32.Serialization.Serialized_Length,
75          Resolver   => Sequence_B_Step_1_Resolver_T'Access)
76 ];
77 Sequence_C_Steps : aliased constant Step_Array :=
78 [
79     0 =>
80         (Kind      => Command_Step,
81          Id        => Component_A_Command_1,
82          Arg_Length => 0,
83          Arg        => [others => 0]),
84     1 =>
85         (Kind      => Command_Step,
86          Id        => Component_A_Command_1,
87          Arg_Length => 0,
88          Arg        => [others => 0])
89 ];
90 Sequence_D_Steps : aliased constant Step_Array :=
91 [
92     0 =>
93         (Kind      => Runtime_Sleep,
94          Id        => 0,
95          Arg_Length => Packed_U32.Serialization.Serialized_Length,
96          Sleep_Resolver => Sequence_D_Step_0_Resolver_T'Access)
97 ];
98 Sequence_E_Steps : aliased constant Step_Array :=
99 [

```

```

100     0 =>
101         (Kind      => Command_Step,
102          Id        => Sequencer_Sequence_C,
103          Arg_Length => 0,
104          Arg        => [others => 0]),
105     1 =>
106         (Kind      => Command_Step,
107          Id        => Component_A_Command_1,
108          Arg_Length => 0,
109          Arg        => [others => 0])
110 ];
111 Sequence_F_Steps : aliased constant Step_Array :=
112 [
113     0 =>
114         (Kind      => Command_Step,
115          Id        => Component_A_Command_1,
116          Arg_Length => 0,
117          Arg        => [others => 0])
118 ];
119 Sequence_G_Steps : aliased constant Step_Array :=
120 [
121     0 =>
122         (Kind      => Command_Step,
123          Id        => Component_A_Command_1,
124          Arg_Length => 0,
125          Arg        => [others => 0])
126 ];
127 Sequence_H_Steps : aliased constant Step_Array :=
128 [
129     0 =>
130         (Kind      => Runtime_Argument_Command_Step,
131          Id        => Component_A_Command_8,
132          Arg_Length => Packed_U16.Serialization.Serialized_Length,
133          Resolver   => Sequence_H_Step_0_Resolver_T'Access),
134     1 =>
135         (Kind      => Runtime_Argument_Command_Step,
136          Id        => Component_A_Command_7,
137          Arg_Length => Packed_Nibble.Serialization.Serialized_Length,
138          Resolver   => Sequence_H_Step_1_Resolver_T'Access)
139 ];
140 Sequence_I_Steps : aliased constant Step_Array :=
141 [
142     0 =>
143         (Kind      => Sleep,
144          Id        => 0,
145          Arg_Length => 0,
146          Sleep_Arg  => 3000)
147 ];
148 Sequence_J_Steps : aliased constant Step_Array :=
149 [
150     0 =>
151         (Kind      => Runtime_Argument_Command_Step,
152          Id        => Component_A_Command_3,
153          Arg_Length => Packed_U32.Serialization.Serialized_Length,
154          Resolver   => Sequence_J_Step_0_Resolver_T'Access)
155 ];
156 Sequence_K_Steps : aliased constant Step_Array :=
157 [
158     0 =>
159         (Kind      => Runtime_Sleep,
160          Id        => 0,

```

```

161     Arg_Length      => Packed_U32.Serialization.Serialized_Length,
162     Sleep_Resolver => Sequence_K_Step_0_Resolver_T'Access)
163 ];
164 -----
165 -- Sequences_Table
166 -----
167 Sequences_Table : aliased constant Sequences_Type :=
168 [
169     0 =>
170         (Wait_For_Cmd_Resp    => True,
171          Abort_On_Failed_Cmd => False,
172          Command_Timeout      => Ada.Real_Time.Milliseconds (10000),
173          Response_Behavior    => Sequence_Enums.Sequence_Response_Behavior.Send]
174          ↪ _After_Sequence_Start,
175          Steps                => Sequence_A_Steps'Access),
176     1 =>
177         (Wait_For_Cmd_Resp    => True,
178          Abort_On_Failed_Cmd => True,
179          Command_Timeout      => Ada.Real_Time.Milliseconds (10000),
180          Response_Behavior    => Sequence_Enums.Sequence_Response_Behavior.Send]
181          ↪ _After_Sequence_Start,
182          Steps                => Sequence_B_Steps'Access),
183     2 =>
184         (Wait_For_Cmd_Resp    => False,
185          Abort_On_Failed_Cmd => True,
186          Command_Timeout      => Ada.Real_Time.Milliseconds (10000),
187          Response_Behavior    => Sequence_Enums.Sequence_Response_Behavior.Send]
188          ↪ _After_Sequence_Start,
189          Steps                => Sequence_C_Steps'Access),
190     3 =>
191         (Wait_For_Cmd_Resp    => True,
192          Abort_On_Failed_Cmd => True,
193          Command_Timeout      => Ada.Real_Time.Milliseconds (10000),
194          Response_Behavior    => Sequence_Enums.Sequence_Response_Behavior.Send]
195          ↪ _After_Sequence_Start,
196          Steps                => Sequence_D_Steps'Access),
197     4 =>
198         (Wait_For_Cmd_Resp    => True,
199          Abort_On_Failed_Cmd => True,
200          Command_Timeout      => Ada.Real_Time.Milliseconds (10000),
201          Response_Behavior    => Sequence_Enums.Sequence_Response_Behavior.Send]
202          ↪ _After_Sequence_Start,
203          Steps                => Sequence_E_Steps'Access),
204     5 =>
205         (Wait_For_Cmd_Resp    => False,
206          Abort_On_Failed_Cmd => True,
207          Command_Timeout      => Ada.Real_Time.Milliseconds (10000),
208          Response_Behavior    => Sequence_Enums.Sequence_Response_Behavior.Send]
209          ↪ _After_Sequence_Completion,
210          Steps                => Sequence_F_Steps'Access),
211     6 =>
212         (Wait_For_Cmd_Resp    => True,
213          Abort_On_Failed_Cmd => True,
214          Command_Timeout      => Ada.Real_Time.Milliseconds (10000),

```

```

215         Response_Behavior => Sequence_Enums.Sequence_Response_Behavior.Send,
216         ↳ _After_Sequence_Start,
217     Steps => Sequence_H_Steps'Access),
218 8 =>
219     (Wait_For_Cmd_Resp => True,
220     Abort_On_Failed_Cmd => True,
221     Command_Timeout => Ada.Real_Time.Milliseconds (10000),
222     Response_Behavior => Sequence_Enums.Sequence_Response_Behavior.Send,
223     ↳ _After_Sequence_Start,
224     Steps => Sequence_I_Steps'Access),
225 9 =>
226     (Wait_For_Cmd_Resp => True,
227     Abort_On_Failed_Cmd => True,
228     Command_Timeout => Ada.Real_Time.Milliseconds (10000),
229     Response_Behavior => Sequence_Enums.Sequence_Response_Behavior.Send,
230     ↳ _After_Sequence_Start,
231     Steps => Sequence_J_Steps'Access),
232 10 =>
233     (Wait_For_Cmd_Resp => True,
234     Abort_On_Failed_Cmd => True,
235     Command_Timeout => Ada.Real_Time.Milliseconds (10000),
236     Response_Behavior => Sequence_Enums.Sequence_Response_Behavior.Send,
237     ↳ _After_Sequence_Start,
238     Steps => Sequence_K_Steps'Access)
239 ];
240
241 Sequences : constant Sequences_Access := Sequences_Table'Access;
242
243 -- The instance configuration handed to Simple_Command_Sequencer.Init. The
244 -- frame-pool size comes from this suite's model (num_concurrent_sequences),
245 -- which also sizes the generated
246 ↳ Test_Assembly_Command_Sequences_Example_Sequences_Summary_Record ground
247 ↳ type for
248 -- the summary packet -- so the frame pool and the packet layout can never
249 -- disagree.
250 Config : constant Sequencer_Config :=
251     (Sequences => Sequences_Table'Access,
252     Num_Concurrent_Sequences => 2);
253 end Test_Assembly_Command_Sequences_Example_Sequences;

```

```

1  -- Auto-generated by Adamant command sequences generator.
2  -- Do not edit by hand.
3  with Interfaces;
4  with Sequence_B_Arg.Validation;
5  with Packed_U32.Validation;
6  with Nibble_Test_Args.Validation;
7  with Validated_Args.Validation;
8  package body Test_Assembly_Command_Sequences_Example_Sequences is
9
10     -----
11     -- Resolvers - one per dynamic step. Each validates the sequence's per-call
12     -- argument buffer, deserializes it as the sequence's arg type, and returns
13     -- the serialized leaf field for its step's buffer (a sub-command's
14     -- Arg_Buffer, or a Packed_U32 millisecond count for a dynamic sleep).
15     -----
16     function Sequence_B_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
17     ↳ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
18     Errant_Field : Interfaces.Unsigned_32;
19     Input : Sequence_B_Arg.T;
20     Valid : Boolean := Sequence_B_Arg.Validation.Valid (Bytes (Bytes'First ..
21     ↳ Bytes'First + Sequence_B_Arg.Serialization.Serialized_Length - 1),
22     ↳ Errant_Field);

```

```

20   begin
21       if Valid then
22           Input := Sequence_B_Arg.Serialization.From_Byte_Array (Bytes
                ↪ (Bytes'First .. Bytes'First +
                ↪ Sequence_B_Arg.Serialization.Serialized_Length - 1));
23           Args := To_Arg (Packed_U32.Serialization.To_Byte_Array
                ↪ (Input.Component_A_Arg));
24       end if;
25       return Valid;
26   end Sequence_B_Step_0_Resolver_T;
27   function Sequence_B_Step_1_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
                ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
28       Errant_Field : Interfaces.Unsigned_32;
29       Input : Sequence_B_Arg.T;
30       Valid : Boolean := Sequence_B_Arg.Validation.Valid (Bytes (Bytes'First ..
                ↪ Bytes'First + Sequence_B_Arg.Serialization.Serialized_Length - 1),
                ↪ Errant_Field);
31   begin
32       if Valid then
33           Input := Sequence_B_Arg.Serialization.From_Byte_Array (Bytes
                ↪ (Bytes'First .. Bytes'First +
                ↪ Sequence_B_Arg.Serialization.Serialized_Length - 1));
34           Args := To_Arg (Packed_U32.Serialization.To_Byte_Array
                ↪ (Input.Component_B_Arg));
35       end if;
36       return Valid;
37   end Sequence_B_Step_1_Resolver_T;
38   function Sequence_D_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
                ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
39       Errant_Field : Interfaces.Unsigned_32;
40       Input : Packed_U32.T;
41       Valid : Boolean := Packed_U32.Validation.Valid (Bytes (Bytes'First ..
                ↪ Bytes'First + Packed_U32.Serialization.Serialized_Length - 1),
                ↪ Errant_Field);
42   begin
43       if Valid then
44           Input := Packed_U32.Serialization.From_Byte_Array (Bytes (Bytes'First
                ↪ .. Bytes'First + Packed_U32.Serialization.Serialized_Length - 1));
45           Args := To_Arg (Packed_U32.Serialization.To_Byte_Array (Input));
46       end if;
47       return Valid;
48   end Sequence_D_Step_0_Resolver_T;
49   function Sequence_H_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
                ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
50       Errant_Field : Interfaces.Unsigned_32;
51       Input : Nibble_Test_Args.T;
52       Valid : Boolean := Nibble_Test_Args.Validation.Valid (Bytes (Bytes'First
                ↪ .. Bytes'First + Nibble_Test_Args.Serialization.Serialized_Length -
                ↪ 1), Errant_Field);
53   begin
54       if Valid then
55           Input := Nibble_Test_Args.Serialization.From_Byte_Array (Bytes
                ↪ (Bytes'First .. Bytes'First +
                ↪ Nibble_Test_Args.Serialization.Serialized_Length - 1));
56           Args := To_Arg (Packed_U16.Serialization.To_Byte_Array (Input.Wide));
57       end if;
58       return Valid;
59   end Sequence_H_Step_0_Resolver_T;
60   function Sequence_H_Step_1_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
                ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
61       Errant_Field : Interfaces.Unsigned_32;

```

```

62     Input : Nibble_Test_Args.T;
63     Valid : Boolean := Nibble_Test_Args.Validation.Valid (Bytes (Bytes'First
    ↪ .. Bytes'First + Nibble_Test_Args.Serialization.Serialized_Length -
    ↪ 1), Errant_Field);
64 begin
65     if Valid then
66         Input := Nibble_Test_Args.Serialization.From_Byte_Array (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Nibble_Test_Args.Serialization.Serialized_Length - 1));
67         Args := To_Arg (Packed_Nibble.Serialization.To_Byte_Array
    ↪ (Input.Nibble));
68     end if;
69     return Valid;
70 end Sequence_H_Step_1_Resolver_T;
71 function Sequence_J_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
    ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
72     Errant_Field : Interfaces.Unsigned_32;
73     Input : Validated_Args.T;
74     Valid : Boolean := Validated_Args.Validation.Valid (Bytes (Bytes'First ..
    ↪ Bytes'First + Validated_Args.Serialization.Serialized_Length - 1),
    ↪ Errant_Field);
75 begin
76     if Valid then
77         Input := Validated_Args.Serialization.From_Byte_Array (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Validated_Args.Serialization.Serialized_Length - 1));
78         Args := To_Arg (Packed_U32.Serialization.To_Byte_Array
    ↪ (Input.Duration));
79     end if;
80     return Valid;
81 end Sequence_J_Step_0_Resolver_T;
82 function Sequence_K_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
    ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
83     Errant_Field : Interfaces.Unsigned_32;
84     Input : Validated_Args.T;
85     Valid : Boolean := Validated_Args.Validation.Valid (Bytes (Bytes'First ..
    ↪ Bytes'First + Validated_Args.Serialization.Serialized_Length - 1),
    ↪ Errant_Field);
86 begin
87     if Valid then
88         Input := Validated_Args.Serialization.From_Byte_Array (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Validated_Args.Serialization.Serialized_Length - 1));
89         Args := To_Arg (Packed_U32.Serialization.To_Byte_Array
    ↪ (Input.Duration));
90     end if;
91     return Valid;
92 end Sequence_K_Step_0_Resolver_T;
93 end Test_Assembly_Command_Sequences_Example_Sequences;
94

```

5 Per-Sequence First-Class Commands

In addition to the sequence tables above, this same model drives the autocoded synthesis of one first-class command per declared sequence on the Simple Command Sequencer instance. Discovery is build-path-driven: the sequencer's commands model queries the model database for every `*.command_sequences.yaml` in scope, so a naked component build (no assembly) sees no sequences and produces only the four static commands (Run_Sequence, Kill_All_Sequences, Kill_Frame, Set_Summary_Packet_Period), while an assembly build whose path includes a

sequences suite adds one command per sequence. Each per-sequence command (e.g. Sequence_B) takes the sequence's own `arg_type` verbatim as its command argument (or no argument for an argless sequence) – a user-written, normally-registered type, so no argument records are generated or registered anywhere in the pipeline. At run time the component copies the received argument buffer unchanged into a generic `Run_Sequence_Arg.T` and dispatches through the existing `Run_Sequence` backbone using the sequence's runtime slot index; the response behavior comes from the sequence's static configuration in the autocoded table (see Sequence Response Behavior below). A companion generated package, `<suite>_Commands`, carries the operator-side builder surface (per-sequence command id getters and `Command.T` constructors) for unit tests and other on-board callers – split from the sequences package because the sequencer component itself never uses the builders. No per-sequence Ada needs to be hand-written:

```

1  -- Auto-generated by Adamant command sequences generator.
2  -- Do not edit by hand.
3  -- Common flight software command sequences
4  --
5  -- Operator-side command builders for the per-sequence ("ghost") commands of
6  -- the Test_Assembly_Command_Sequences_Example_Sequences suite. These commands
7  -- ↪ are first-class in the assembly
8  -- command dictionary but have no generated Ada handler on the component, so
9  -- this package reconstructs the builders (id getters + Command.T
10 -- constructors) for unit tests and other on-board callers. The sequencer
11 -- component itself does not use this package.
12 with Command;
13 with Command_Types;
14 with Nibble_Test_Args;
15 with Packed_U32;
16 with Sequence_B_Arg;
17 with Validated_Args;
18 package Test_Assembly_Command_Sequences_Example_Sequences_Commands is
19
20     -- Set Id_Base to the sequencer instance's command id base (the value passed
21     -- to Set_Id_Bases). Each sequence command id is then
22     -- Id_Base + Simple_Command_Sequencer_Commands.Num_Commands + declaration
23     -- index.
24     type Instance is tagged private;
25
26     -- Set the sequencer instance's command id base so constructed commands
27     -- ↪ carry
28     -- ids that match what the component registered at runtime.
29 procedure Set_Id_Base (Self : in out Instance; Id_Base : in
30     ↪ Command_Types.Command_Id);
31
32     -- Set the operator source id stamped into the header of constructed commands
33     -- (defaults to 0). Mirrors the command suite's Set_Source_Id.
34 procedure Set_Source_Id (Self : in out Instance; Source_Id : in
35     ↪ Command_Types.Command_Source_Id);
36
37 function Get_Sequence_A_Id (Self : in Instance) return
38     ↪ Command_Types.Command_Id;
39 function Sequence_A (Self : in Instance) return Command.T;
40 function Get_Sequence_B_Id (Self : in Instance) return
41     ↪ Command_Types.Command_Id;
42 function Sequence_B (Self : in Instance; Arg : in Sequence_B_Arg.T) return
43     ↪ Command.T;
44 function Get_Sequence_C_Id (Self : in Instance) return
45     ↪ Command_Types.Command_Id;
46 function Sequence_C (Self : in Instance) return Command.T;
47 function Get_Sequence_D_Id (Self : in Instance) return
48     ↪ Command_Types.Command_Id;
49 function Sequence_D (Self : in Instance; Arg : in Packed_U32.T) return
50     ↪ Command.T;

```

```

41  function Get_Sequence_E_Id (Self : in Instance) return
    ↪ Command_Types.Command_Id;
42  function Sequence_E (Self : in Instance) return Command.T;
43  function Get_Sequence_F_Id (Self : in Instance) return
    ↪ Command_Types.Command_Id;
44  function Sequence_F (Self : in Instance) return Command.T;
45  function Get_Sequence_G_Id (Self : in Instance) return
    ↪ Command_Types.Command_Id;
46  function Sequence_G (Self : in Instance) return Command.T;
47  function Get_Sequence_H_Id (Self : in Instance) return
    ↪ Command_Types.Command_Id;
48  function Sequence_H (Self : in Instance; Arg : in Nibble_Test_Args.T) return
    ↪ Command.T;
49  function Get_Sequence_I_Id (Self : in Instance) return
    ↪ Command_Types.Command_Id;
50  function Sequence_I (Self : in Instance) return Command.T;
51  function Get_Sequence_J_Id (Self : in Instance) return
    ↪ Command_Types.Command_Id;
52  function Sequence_J (Self : in Instance; Arg : in Validated_Args.T) return
    ↪ Command.T;
53  function Get_Sequence_K_Id (Self : in Instance) return
    ↪ Command_Types.Command_Id;
54  function Sequence_K (Self : in Instance; Arg : in Validated_Args.T) return
    ↪ Command.T;
55
56  private
57      type Instance is tagged record
58          Id_Base : Command_Types.Command_Id := 0;
59          Source_Id : Command_Types.Command_Source_Id := 0;
60      end record;
61  end Test_Assembly_Command_Sequences_Example_Sequences_Commands;

```

6 Sequences Calling Sequences

Because every sequence is a first-class command on the sequencer instance, a sequence step may invoke another sequence simply by targeting the sequencer's own synthesized command, e.g. `command: My_Sequencer.Sequence_C`. No special syntax or generator support is involved – the step resolves like any other `Component.Command` reference, provided the sequencer instance is declared in the assembly. At run time the sub-sequence command is emitted out the sequencer's `Command_T_Send` connector, routed through the command router, and arrives back at the sequencer's `Command_T_Recv_Async`, where it claims its own frame and executes concurrently with (and independently of) the calling sequence. If the calling sequence waits for command completion, the sub-sequence's command response (per its static `response_behavior` configuration, see below) is what wakes it. Nesting depth is naturally capped by `Num_Concurrent_Sequences`: a sub-sequence call when no frame is available is rejected with a `No_Frame_Available` event and a `Failure` response to the calling frame.

7 Sequence Response Behavior

When the sequencer replies to the command that started a sequence is controlled by the `Sequence_Response_Behavior` enumeration. For the synthesized per-sequence commands it is static configuration: the optional `response_behavior` field on each sequence in the YAML, baked into the autocoded sequence table. The generic `Run_Sequence` command instead carries the behavior as an argument, so a per-call override is always available through it:

- `Send_After_Sequence_Start` (the conventional default) – the command response is sent immediately once the sequence has been accepted and started on a frame. This is the right

choice for fire-and-forget operation and for sub-sequence calls where the caller should proceed as soon as the sub-sequence is running.

- `Send_After_Sequence_Completion` – the command response is withheld until the sequence ends, and its status reflects the outcome: `Success` when the final step completes, or `Failure` when the sequence is aborted by a failed sub-command, times out waiting for a sub-command response, fails to resolve or validate a dynamic argument, encounters an out-of-range sleep or timeout, or is halted by `Kill_All_Sequences` or `Kill_Frame`. The sequencer captures the originating command's source and id per frame, so concurrent deferred sequences each reply to their own operator.

A sequence step may forward a field of the parent sequence's argument to the sub-command it dispatches, using the `Arg.field.subfield` syntax in the step's `arg:`. Each such reference produces a typed Resolver in the generated package: at run time the sequencer dispatches that Resolver, which first validates the per-call argument buffer through the input type's generated `Validation.Valid` (a failure aborts the sequence with an `Invalid_Dynamic_Command_Argument` event), then deserializes it through the input type's safe `Serialization.From_Byte_Array`, navigates to the addressed field, and re-serializes it as the sub-command's argument buffer. The resolver's signature accepts only a `Basic_Types.Byte_Array` – there is no 'Address reinterpretation' – so the dynamic-argument machinery is type-safe by construction; a resolver cannot overlay an arbitrary record onto raw memory, only deserialize a byte array through the input type's generated deserializer.

8 Sleep Steps

A sequence step may be a sleep instead of a command: `sleep_ms:` pauses the executing frame without issuing any bus traffic. Sleep is not a bus command; it is interpreted intrinsically by the sequencer frame state machine. The duration takes two forms. A *static* sleep (`sleep_ms: 3000`) is a plain integer number of milliseconds fixed at build time; the model bounds it to $0 \dots 2^{31} - 1$ so it always fits an `Ada.Real_Time.Time_Span` by construction, leaving wake-time overflow of `Sys_Time` arithmetic as its only run-time failure. A *dynamic* sleep (`sleep_ms: Arg.Settle_Time`) instead references the sequence's per-call argument with the same `Arg[.field]` syntax as dynamic command arguments; the referenced field must be a `Packed_U32.T`, resolved at execution time through the same validating Resolver machinery (a validation failure aborts the sequence with an `Invalid_Dynamic_Sleep_Argument` event) and range-checked before scheduling. A sleep that fails its range check or overflows the wake-time computation ends the sequence with a `Sequence_Out_Of_Range_Sleep` event and a `Failure` status to any pending deferred reply – continuing would leave the frame parked on a stale wake time.

9 Summary Packet Type

The sequencer emits a periodic `Summary_Packet` containing one `Sequence_Frame_Summary` record per sequence frame, in frame order. Its layout therefore depends on the suite's `num_concurrent_sequences`, so the generator also autocodes the packet's ground/documentation record type, `<suite>_summary_record.record.yaml`, alongside the suite's other outputs. The flight software never imports this type – the component fills the packet buffer directly from its frame array – but ground tools and assembly documentation need it. At assembly load time the sequencer's packets model resolves an instance's `Summary_Packet` type to the summary record of the suite named by the instance's `Config init` parameter. Because the frame pool and the packet layout both come from the suite model's single `num_concurrent_sequences` value, they can never disagree; the model also rejects at build time any suite whose frame summaries would not fit in one packet buffer (the same bound the `Num_Concurrent_Sequences_Type` subtype enforces at `Init`). The example input above generates the following record model:

```
1 ---
2 description: Autocoded summary packet type for the
   ↳ Test_Assembly_Command_Sequences_Example_Sequences command sequences suite.
   ↳ Contains one Sequence_Frame_Summary record per sequence frame (2 total, from
   ↳ the suite's num_concurrent_sequences), in frame order.
```

```
3 fields:
4   - name: Frame_0_Summary
5     description: "A summary of the state of sequence frame 0."
6     type: Sequence_Frame_Summary.T
7   - name: Frame_1_Summary
8     description: "A summary of the state of sequence frame 1."
9     type: Sequence_Frame_Summary.T
10
```